Nutrition Tracker

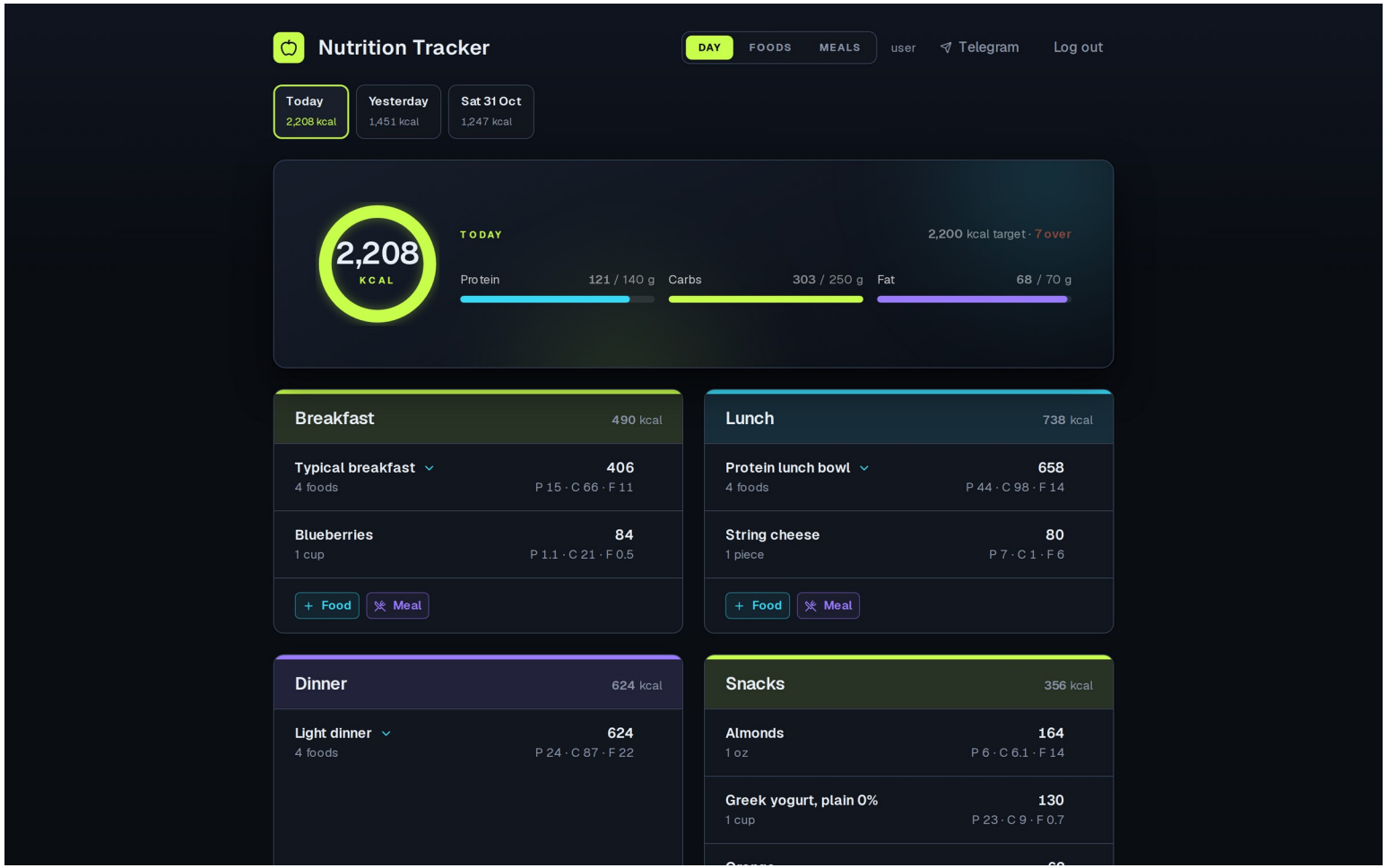
A food log I use every day.

Built end to end with AI agents
— and directed as an
engineering project.

What it is

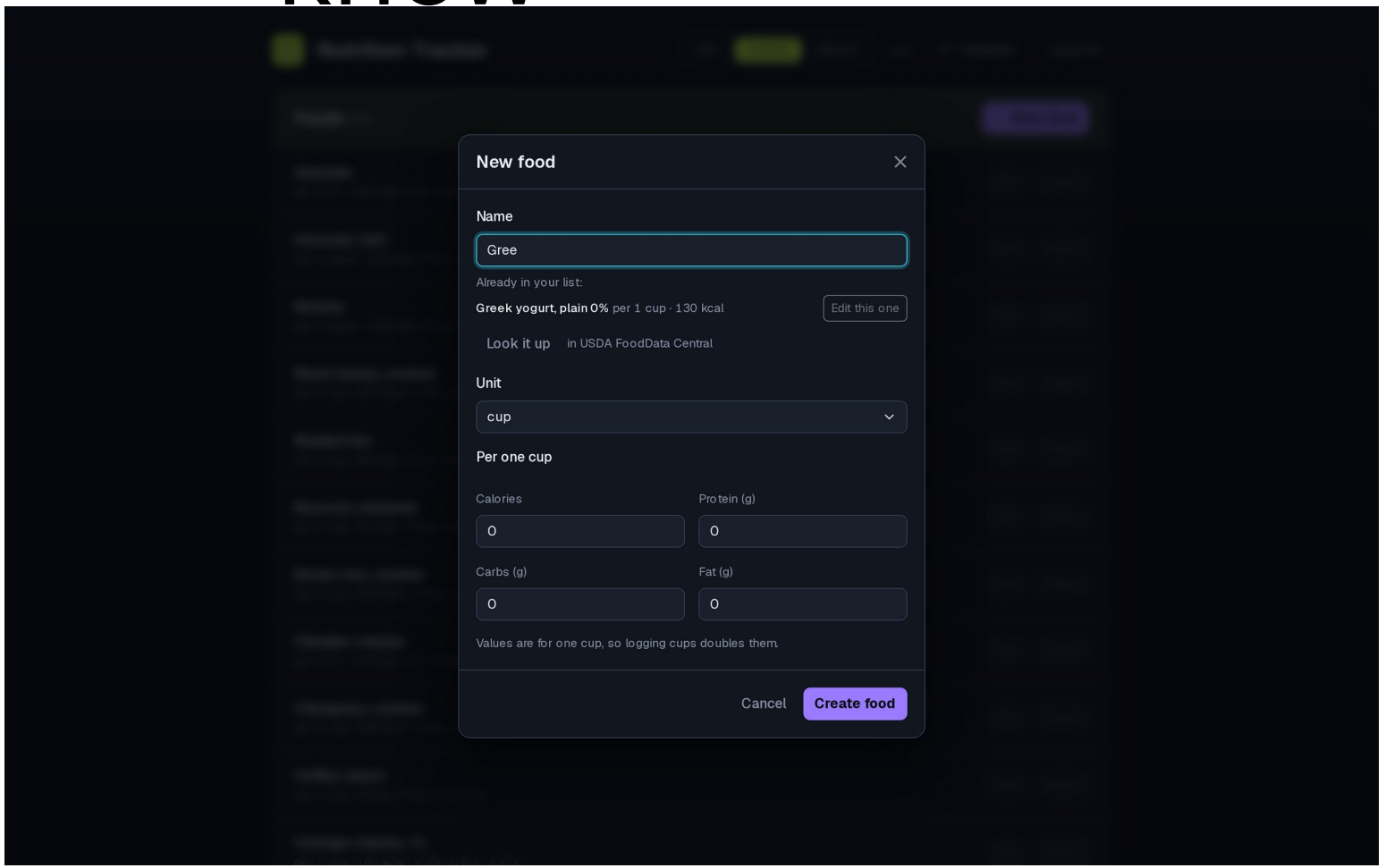
- A daily food and macro log: what I ate, what it came to, and what is left against my targets
- Next.js and TypeScript front end, Python (FastAPI) API, SQLite, Docker — my own project
- A Telegram bot as a second client of the same API, so it works from a phone
- Multiple accounts, isolated from each other at the API and in the database
- No model in the product: AI wrote most of the code, the app is deterministic code

The day view



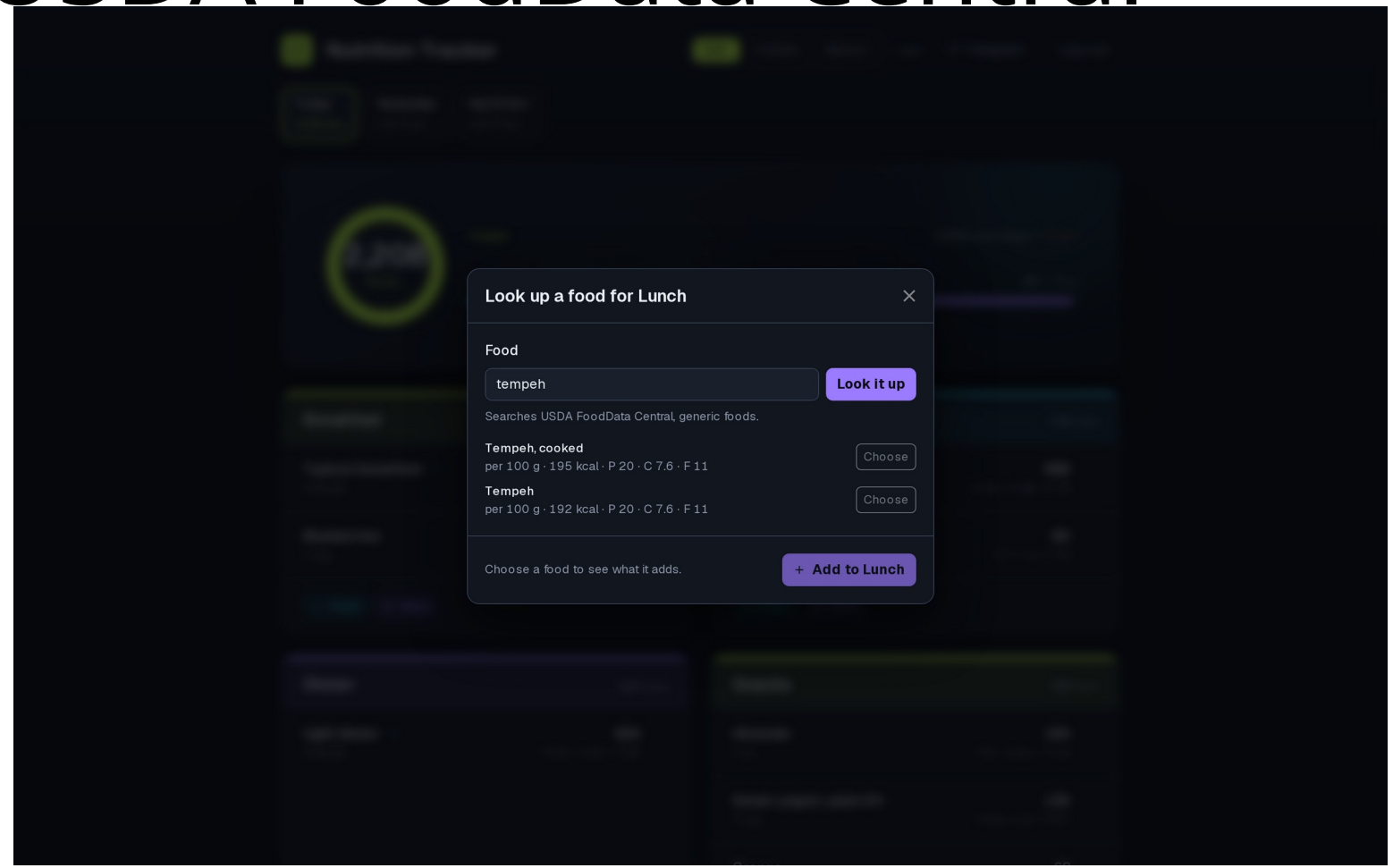
Opens on the newest day: sections, entries, running totals, and macro bars against target.

Adding a food you already know



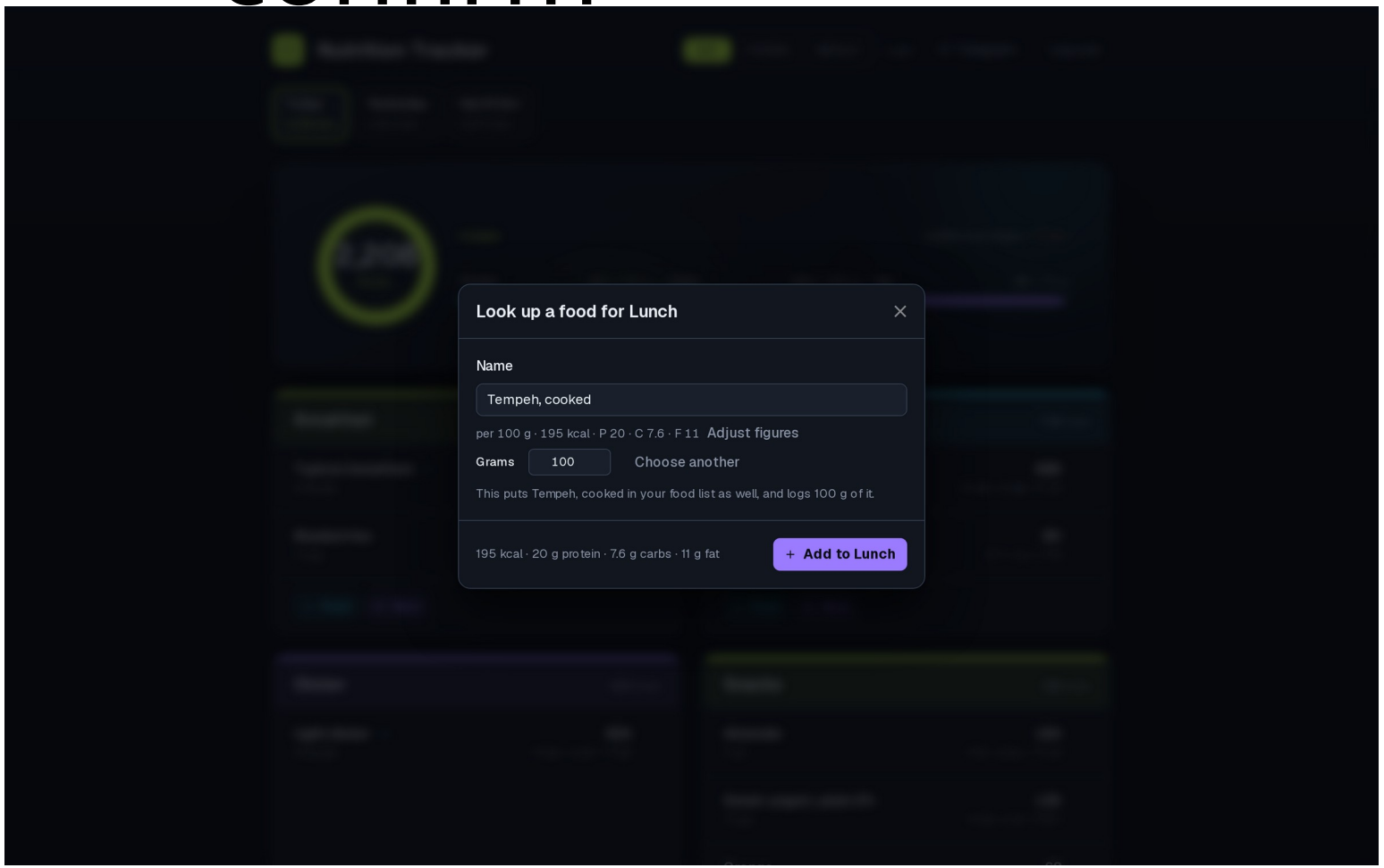
A near-duplicate already in the list is offered instead of silently created — the guard against phantom rows.

Foods the catalogue does not have: USDA FoodData Central



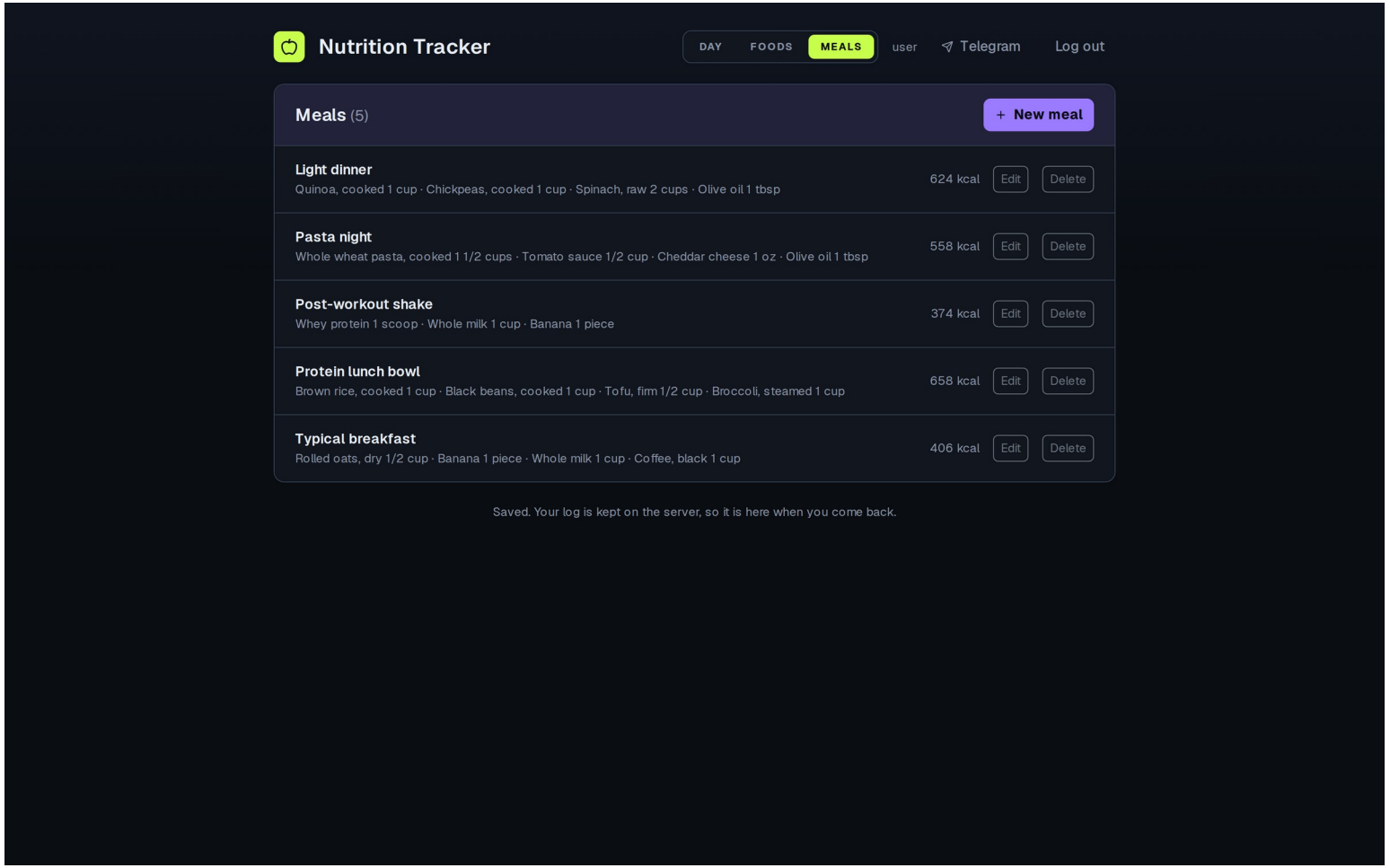
A live USDA search: generic foods only, up to three candidates, each with per-100 g figures.

Nothing is written until you confirm



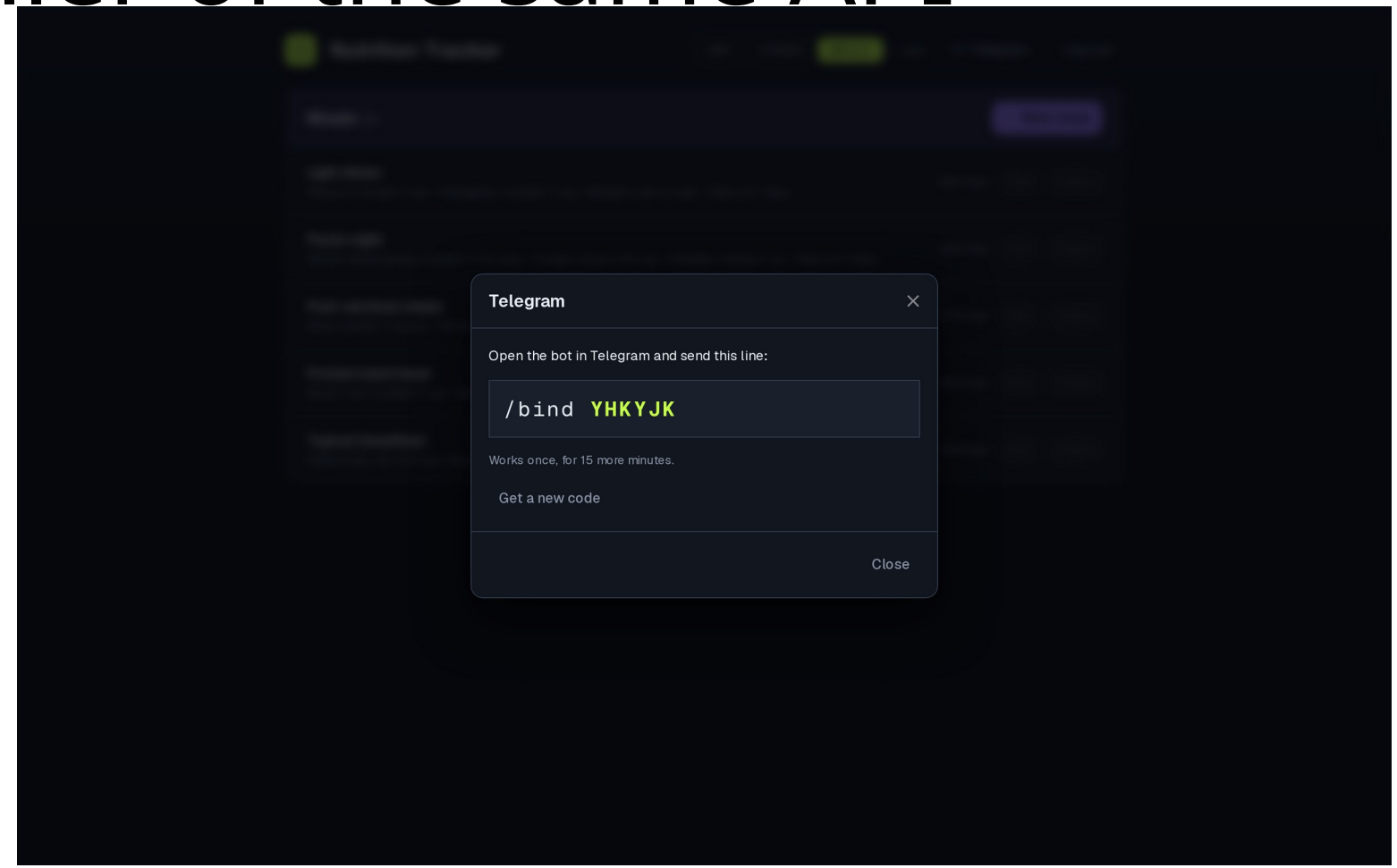
Per 100 g in, grams out: name and figures stay editable, and one button adds it to the catalogue and logs it.

Meals as shortcuts



A named meal logs all of its items in one step. A definition is a snapshot, so a shortcut cannot drift.

The chat client — a second caller of the same API

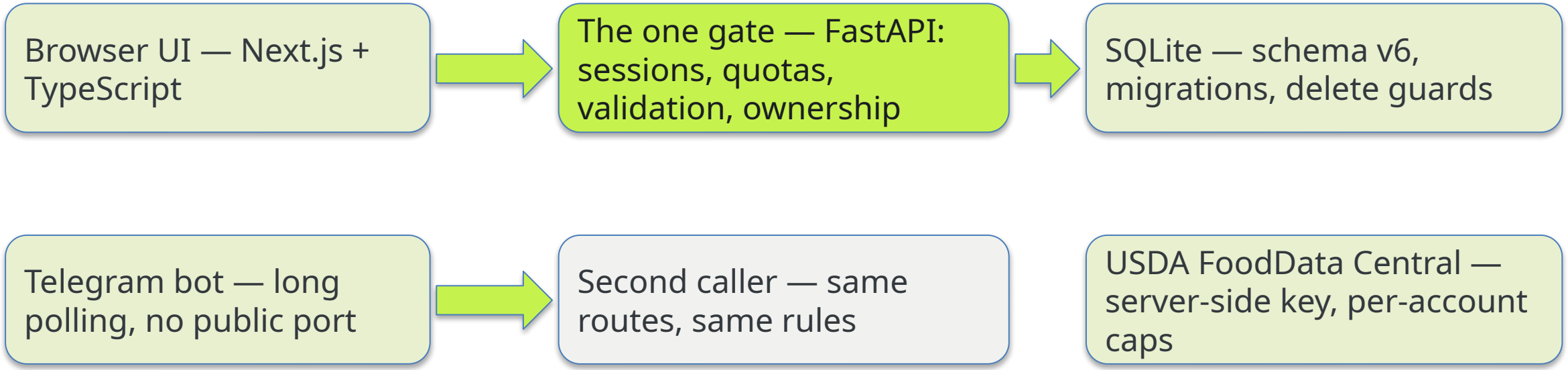


Linking a chat means minting a one-time code and sending it to the bot. Not one password, ever.

No password ever goes into a chat

- Minted by the app, redeemed in the chat, burned on use — retiring any other code that account held
- The bot proves who it is with a shared secret compared in constant time
- The account it acts for comes from the binding, not from the caller
- One function decides who may ask, for person and machine alike
- Per-account daily caps, per-chat attempt limits, codes that expire in fifteen minutes
- Account isolation is proved by tests, not asserted

How it fits together



One API, two callers. The person and the bot authenticate to the same gate, so a route the person cannot reach is a route the bot cannot reach — and an unbound chat is refused everything. The nutrition figures come from USDA and the person's own list, never from a model: the model that built the app is not part of the app.

Data, and how it changes shape safely

- SQLite at schema version 6, with a migration chain that refuses a schema it cannot carry forward
- Deletes are guarded in the database, so an entry cannot lose the food underneath it
- Entries carry literal figures: correcting a value never rewrites what a day was logged as
- Every account's rows are its own — enforced at the API, and proved by tests

How it was built: the process, not a long chat

- A written brief: 30 numbered requirements and 24 recorded decisions, each with its reasoning
- A plan file stating every part's substeps, its tests and its success criteria
- A review gate at every part — nothing starts before the previous part is verified
- The state after each part written down for whoever picks it up next
- Every claim in the documentation is a command that prints it

How it was built: the agents

- AI agents wrote most of the code; I set the requirements, the boundaries and the gates
- Separate agents on isolated profiles, each with only the tools its own job needs
- Private health and training data stays on a model I run myself
- Unknown values are refused rather than guessed; credentials carry the least access that works
- The result is not a demo: it is the app I log my own meals in

Evidence, and how to check it

Suite	Command	Result
Backend (pytest)	uv run python -m pytest	282 passed
Front-end unit (Vitest)	npm test	140 passed, 14 files
End-to-end (Playwright)	npm run test:e2e	28 passed, real container
Schema version	db.py SCHEMA_VERSION	6

Every figure here came from re-running the command, not from memory. The end-to-end suite starts its own container from a freshly seeded database, so it asserts against a known state rather than whatever happens to be on the machine.

What it does not do yet

- No voice input: the bot skips any update with no text field — left off until it exists
- One person, one machine: there is no hosting story yet
- Household units are partial: grams, cups, each, tablespoons
- Rate limits live in memory and reset with the process
- Next: a second account on real hardware, then hosting, then voice